MODUL 19 · BAB VI · ALGORITMA GRAF

Konsep Minimum Spanning Tree (MST)

Menghubungkan semua simpul dengan total bobot sekecil mungkin: definisi, sifat, dan dasar teoretis algoritma Kruskal dan Prim.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan arti spanning tree dan minimum spanning tree pada graf tak berarah berbobot.
- Menyebutkan sifat-sifat tree yang dipakai dalam algoritma MST.
- Menjelaskan konsep cut, crossing edge, light edge, dan safe edge, serta teorema yang menjamin kebenaran algoritma MST.
- Menjelaskan mengapa MST bisa lebih dari satu dan menunjukkan dua MST pada contoh bahan kuliah.
- Membandingkan strategi Kruskal dan Prim secara garis besar.

PRASYARAT

- Modul 16: graf, graf berbobot, dan tree.
- Modul 13: konsep Greedy.

BAB / MATERI ASAL

Materi 12. MINIMUM SPANNING TREE, algoritma Kruskal dan Prim (bagian pengantar)

POSISI DALAM URUTAN BELAJAR

Konsep umum MST. Diterapkan oleh algoritma Kruskal (Modul 20) dan Prim (Modul 21).

DAFTAR ISI

1. Kabel Termurah untuk Menghubungkan Semua Kota 1
2. Spanning Tree dan Minimum Spanning Tree 2
3. Dasar Teoretis: Cut dan Safe Edge 3
4. MST Tidak Selalu Tunggal 4
5. Keterkaitan Konsep 5
6. Kesimpulan 6
- Kuis Pemahaman 7
- Kunci Jawaban dan Pembahasan 8
- Referensi 8

PENDAHULUAN

1. Kabel Termurah untuk Menghubungkan Semua Kota

Sebuah perusahaan ingin menghubungkan sembilan kota dengan jaringan kabel. Biaya pemasangan kabel antara setiap pasangan kota sudah diketahui, tetapi tidak semua pasangan dapat dihubungkan langsung. Syaratnya, setiap kota harus dapat berkomunikasi dengan kota lain (langsung atau lewat kota perantara), dan total biaya harus **sekecil mungkin**.

Jaringan termurah seperti itu tidak akan pernah memuat **cycle**: jika ada cycle, salah satu kabel di cycle tersebut dapat dilepas tanpa memutus komunikasi, sehingga biaya berkurang. Jadi solusinya selalu berbentuk **tree** yang menjangkau semua kota. Masalah ini disebut masalah **Minimum Spanning Tree** (MST).

Bahan kuliah memberikan dua algoritma untuk membangun MST: **Kruskal** dan **Prim**. Keduanya algoritma Greedy (Modul 13). Modul ini membahas konsep yang menjadi dasar keduanya: definisi, sifat tree, dan teorema yang menjelaskan mengapa pilihan greedy di kedua algoritma itu selalu aman.

PEMBAHASAN INTI

2. Spanning Tree dan Minimum Spanning Tree

Misalkan $G = (V, E)$ adalah graf **tak berarah**, **terhubung**, dan **berbobot**: setiap busur (u, v) memiliki bobot $w(u, v)$.

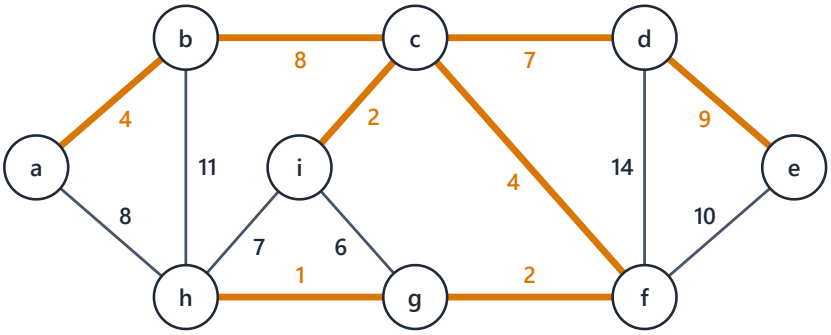
DEFINISI

Spanning tree dari G adalah himpunan busur $T \subseteq E$ yang membentuk tree dan menghubungkan **semua** simpul di V. Bobotnya adalah $w(T) = \sum_{(u,v)\in T} w(u, v)$.

Minimum spanning tree (MST) adalah spanning tree dengan bobot total **minimum**.

2.1 Contoh dari bahan kuliah

Graf tak berarah berbobot dengan 9 simpul dan 14 busur:



Gambar 1. Graf contoh. Busur tebal oranye adalah MST A = {(a,b), (b,c), (c,d), (c,f), (c,i), (d,e), (f,g), (g,h)} dengan bobot total 37.

Himpunan busur $A = \{(a, b), (b, c), (c, d), (c, f), (c, i), (d, e), (f, g), (g, h)\}$ menghubungkan semua 9 simpul tanpa cycle, dengan bobot total $4 + 8 + 7 + 4 + 2 + 9 + 2 + 1 = 37$. Ini adalah MST (bobot 37 adalah yang terkecil yang mungkin).

2.2 Sifat-sifat tree yang dipakai

1. Tree dengan $|V|$ simpul memiliki tepat $|V| - 1$ **busur**. Pada contoh: $9 - 1 = 8$ busur.
2. Pada tree **tidak ada cycle** (lintasan tertutup).
3. Menambahkan satu busur baru ke tree selalu menghasilkan **tepat satu cycle**.
4. Menghapus satu busur dari tree selalu memecah tree menjadi **dua komponen** yang terpisah.

2.3 Mengapa tidak mencoba semua spanning tree?

Banyaknya spanning tree tumbuh sangat cepat. Graf contoh yang kecil ini saja memiliki **662** spanning tree yang berbeda (dihitung dengan program pada bagian Contoh). Untuk graf lengkap dengan n simpul,

banyaknya spanning tree adalah n^{n-2} (rumus Cayley); untuk $n = 20$, itu lebih dari $2{,}6 \times 10^{23}$. Algoritma Kruskal dan Prim menemukan MST dalam waktu $O(E \lg V)$ tanpa mencoba semuanya.

PEMBAHASAN INTI

3. Dasar Teoretis: Cut dan Safe Edge

Kruskal dan Prim sama-sama membangun MST dengan menambahkan busur satu per satu ke sebuah himpunan A, dengan menjaga satu sifat: A **selalu merupakan bagian dari suatu MST**. Busur yang dapat ditambahkan tanpa merusak sifat itu disebut **safe edge**. Cormen dkk. (2009, Bab 23.1) merumuskan algoritma umumnya:

GENERIC-MST(G, w)

```
1  A = ∅
2  while A belum membentuk spanning tree
3      cari busur (u, v) yang safe untuk A
4      A = A ∪ {(u, v)}
5  return A
```

Pertanyaannya: bagaimana mengenali safe edge? Untuk itu dibutuhkan beberapa istilah.

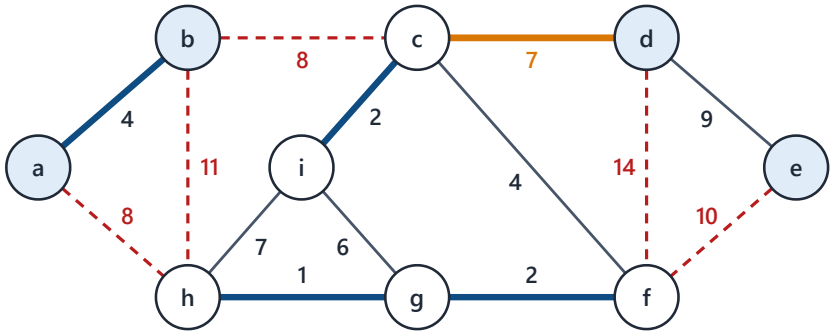
Istilah	Arti
Cut $(S, V - S)$	Pembagian himpunan simpul menjadi dua bagian: S dan sisanya $V - S$.
Crossing edge	Busur yang satu ujungnya di S dan ujung lainnya di $V - S$.
Cut **respects** A	Tidak ada busur di A yang menyeberangi cut tersebut.
Light edge	Crossing edge dengan bobot paling kecil di antara semua crossing edge pada cut itu.

Tabel 1. Istilah untuk mengenali safe edge.

TEOREMA (CORMEN DKK., 2009, TEOREMA 23.1)

Misalkan A adalah himpunan busur yang termuat di suatu MST dari G, dan $(S, V - S)$ adalah sembarang cut yang respects A. Jika (u, v) adalah **light edge** yang menyeberangi cut tersebut, maka (u, v) adalah **safe edge** untuk A.

Gagasan bukti (argumen pertukaran). Misalkan T adalah MST yang memuat A tetapi tidak memuat (u, v). Menambahkan (u, v) ke T membentuk tepat satu cycle (sifat 3). Cycle itu menyeberangi cut dua kali, jadi ada busur lain (x, y) di cycle yang juga menyeberangi cut. Karena (u, v) adalah light edge, $w(u, v) \leq w(x, y)$. Tukar (x, y) dengan (u, v): hasilnya tetap spanning tree, bobotnya tidak bertambah, dan tetap memuat A (karena cut respects A, busur (x, y) bukan anggota A). Jadi ada MST yang memuat $A \cup \{(u, v)\}$.



Gambar 2. Contoh cut $(S, V - S)$ dengan $S = \{a, b, d, e\}$ (simpul biru). Busur merah putus-putus adalah crossing edge; busur tebal biru adalah $A = \{(a, b), (c, i), (f, g), (g, h)\}$, yang tidak menyeberangi cut. Light edge-nya adalah (c, d) berbobot 7, sehingga (c, d) safe.

3.1 Bagaimana Kruskal dan Prim memakai teorema ini

Aspek	Kruskal (Modul 20)	Prim (Modul 21)
Bentuk A selama berjalan	**Hutan** (beberapa tree yang terpisah)	**Satu tree** yang terus tumbuh dari simpul akar r
Pilihan greedy	Busur berbobot terkecil di seluruh graf yang menghubungkan dua tree berbeda	Busur berbobot terkecil yang menghubungkan tree dengan simpul di luar tree
Cut yang dipakai	Antara tree yang memuat u dan sisanya	Antara simpul di dalam tree dan di luar tree
Struktur data utama	Himpunan saling lepas (MAKE-SET, FIND-SET, UNION)	Antrean prioritas minimum (EXTRACT-MIN)
Running time	$O(E \lg V)$	$O(E \lg V)$ dengan binary heap

Tabel 2. Kruskal dan Prim sebagai penerapan GENERIC-MST.

CONTOH DAN PENERAPAN

4. MST Tidak Selalu Tunggal

Graf contoh memiliki dua busur yang sama-sama berbobot 8: (a, h) dan (b, c). Akibatnya MST-nya ada **dua**, keduanya berbobot 37:

MST	Busur	Bobot
T_1 (bahan kuliah)	(a,b), **(b,c)**, (c,d), (c,f), (c,i), (d,e), (f,g), (g,h)	37
T_2	(a,b), **(a,h)**, (c,d), (c,f), (c,i), (d,e), (f,g), (g,h)	37

Tabel 3. Dua MST pada graf contoh.

Keduanya hanya berbeda satu busur. Pada T_1, simpul a terhubung ke bagian kanan lewat $b \to c$; pada T_2 lewat h. Algoritma mana pun yang benar akan menghasilkan salah satunya, tergantung cara memutuskan ketika ada bobot yang sama. Secara umum, jika **semua bobot busur berbeda**, MST pasti tunggal.

Python: memeriksa semua spanning tree secara brute force

```python
from itertools import combinations

E = [("a","b",4), ("a","h",8), ("b","c",8), ("b","h",11), ("c","d",7),
     ("c","i",2), ("c","f",4), ("d","e",9), ("d","f",14), ("e","f",10),
     ("f","g",2), ("g","h",1), ("g","i",6), ("h","i",7)]
V = "abcdefghi"

def spanning_tree(busur):                # |V|-1 busur tanpa cycle = spanning tree
    induk = {v: v for v in V}
    def akar(x):
        while induk[x] != x:
            x = induk[x]
        return x
    for u, v, _ in busur:
        ru, rv = akar(u), akar(v)
        if ru == rv:
            return False                 # membentuk cycle
        induk[ru] = rv
    return True

semua = [t for t in combinations(E, len(V) - 1) if spanning_tree(t)]
bobot = [sum(w for *_, w in t) for t in semua]
print("banyak spanning tree:", len(semua))
print("bobot minimum:", min(bobot), " banyak MST:", bobot.count(min(bobot)))
print("bobot maksimum:", max(bobot))
```

Keluaran program

```
banyak spanning tree: 662
bobot minimum: 37  banyak MST: 2
bobot maksimum: 71
```

Brute force ini memeriksa $\binom{14}{8} = 3.003$ himpunan busur. Untuk graf kecil hal itu masih mungkin, tetapi pertumbuhannya eksponensial, sehingga tidak praktis untuk graf nyata.

ANALISIS DAN HUBUNGAN KONSEP

5. Keterkaitan Konsep

MST adalah masalah Greedy yang terbukti benar. Teorema light edge adalah bentuk khusus greedy-choice property (Modul 13): memilih busur termurah yang menyeberangi cut tidak pernah menutup jalan ke solusi optimal. Bukti dengan argumen pertukaran di atas mirip dengan bukti pada Activity Selection (Modul 14).

MST bukan lintasan terpendek. MST meminimalkan total bobot seluruh jaringan, bukan jarak antara dua simpul tertentu. Pada contoh, lintasan dari a ke h di dalam MST T_1 adalah $a \to b \to c \to f \to g \to h$ dengan bobot $4 + 8 + 4 + 2 + 1 = 19$, padahal busur langsung (a, h) hanya berbobot 8. Masalah lintasan terpendek dibahas tersendiri di Modul 22–24.

Graf harus tak berarah dan terhubung. Jika graf tidak terhubung, tidak ada spanning tree; yang dapat dibentuk adalah *minimum spanning forest*, satu tree untuk setiap komponen. Kruskal akan menghasilkan

forest tersebut secara alami.

KESIMPULAN

6. Kesimpulan

- Spanning tree adalah tree yang menghubungkan semua simpul graf; MST adalah spanning tree dengan total bobot minimum dan memiliki $|V| - 1$ busur.
- Algoritma MST membangun himpunan A dengan menambahkan safe edge; light edge yang menyeberangi cut yang respects A selalu safe (Teorema 23.1).
- Kruskal menumbuhkan forest dengan memilih busur termurah antar-tree; Prim menumbuhkan satu tree dengan memilih busur termurah yang keluar dari tree.
- MST bisa lebih dari satu bila ada bobot yang sama; graf contoh memiliki 662 spanning tree dan dua MST berbobot 37.
- MST meminimalkan total bobot jaringan, bukan jarak antara dua simpul.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Sebuah graf terhubung memiliki 12 simpul dan 30 busur. Berapa banyak busur pada MST-nya?

A. 11
B. 12
C. 29
D. 30

2 ANALISIS

Jelaskan mengapa solusi masalah "jaringan kabel termurah yang menghubungkan semua kota" tidak mungkin memuat cycle (asumsikan semua biaya positif).

3 PENERAPAN TEOREMA

Pada graf contoh, perhatikan cut $S = \{a\}$ dan $V - S$ = simpul lainnya, dengan $A = \emptyset$. Apa saja crossing edge-nya, dan busur mana yang dijamin safe?

4 BENAR/SALAH DENGAN ALASAN

"Jika bobot busur sebuah graf terhubung semuanya berbeda, MST-nya tunggal." Benar atau salah?

5 ANALISIS KASUS

Pada graf contoh, bobot busur (b, c) diubah dari 8 menjadi 9. Bagaimana pengaruhnya terhadap MST dan bobot totalnya?

6 PILIHAN GANDA

Perbedaan utama Kruskal dan Prim selama algoritma berjalan adalah ...

A. Kruskal hanya untuk graf berarah.
B. Kruskal memelihara forest yang lama-kelamaan menyatu; Prim memelihara satu tree yang terus tumbuh.
C. Prim tidak bersifat greedy.
D. Kruskal selalu menghasilkan bobot yang lebih kecil.

7 ANALISIS

Tunjukkan dengan contoh dari graf bahan kuliah bahwa lintasan antara dua simpul di dalam MST belum tentu lintasan terpendek antara kedua simpul tersebut.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: A. MST adalah tree yang menjangkau semua 12 simpul, sehingga memiliki $12 - 1 = 11$ busur, berapa pun banyaknya busur graf semula.

Soal 2

Jika jaringan memuat cycle, salah satu kabel di cycle itu dapat dilepas dan semua kota tetap terhubung, karena dua ujung kabel tersebut masih terhubung lewat sisa cycle. Biaya totalnya berkurang (biaya positif), sehingga jaringan semula bukan yang termurah. Jadi jaringan termurah tidak memiliki cycle, yaitu berbentuk tree.

Soal 3

Crossing edge: (a, b) berbobot 4 dan (a, h) berbobot 8. Cut ini respects $A = \emptyset$, dan light edge-nya adalah (a, b), sehingga (a, b) dijamin safe, artinya ada MST yang memuatnya. (Inilah langkah pertama Prim dengan akar a.)

Soal 4

Benar. Ini sifat yang dikenal luas (lihat latihan pada Cormen dkk., 2009, Bab 23.1). Ketidaktunggalan MST hanya dapat terjadi bila ada busur berbobot sama yang dapat saling menggantikan, seperti (a, h) dan (b, c) pada contoh.

Soal 5

MST T_1 (yang memakai (b, c)) bobotnya menjadi 38, sedangkan T_2 (yang memakai (a, h)) tetap 37. Jadi MST-nya menjadi **tunggal**, yaitu T_2, dengan bobot tetap **37**.

Soal 6

Jawaban: B. Keduanya untuk graf tak berarah (A salah), keduanya greedy (C salah), dan keduanya menghasilkan MST dengan bobot minimum yang sama (D salah).

Soal 7

Di MST T_1, lintasan dari a ke h adalah $a \to b \to c \to f \to g \to h$ dengan bobot $4 + 8 + 4 + 2 + 1 = 19$. Padahal busur langsung (a, h) berbobot 8. Jadi lintasan di dalam MST tidak harus lintasan terpendek.

Referensi

Sumber utama: Materi 12. MINIMUM SPANNING TREE, algoritma Kruskal dan Prim (bagian pengantar) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Kruskal, J. B. (1956). On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem. *Proceedings of the American Mathematical Society*, 7(1), 48–50. https://doi.org/10.1090/S0002-9939-1956-0078686-7
3. Prim, R. C. (1957). Shortest Connection Networks and Some Generalizations. *Bell System Technical Journal*, 36(6), 1389–1401. https://doi.org/10.1002/j.1538-7305.1957.tb01515.x

4. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.